



ApexaiQ Technology Asset Management

Documentation

Weekly Research Report

Yash Ramesh Fating | Date : 06-mar-2026

1. Variables

Concept

Variables are containers used to store data values in a program. JavaScript provides three main ways to declare variables:

- **var**
- **let**
- **const**

The modern best practice is to use **let** and **const**.

Keyword	Scope	Can Reassign	Can Redeclare
var	Function	Yes	Yes
let	Block	Yes	No
const	Block	No	No

Practical Program

```
// Variable Declaration
```

```
var name = "Yash";  
let age = 21;
```

```
const country = "India";

console.log("Name:", name);
console.log("Age:", age);
console.log("Country:", country);

// Reassigning variable
age = 22;
console.log("Updated Age:", age);
```

Explanation

In this program:

- **var name** stores a string value.
- **let age** can be updated later.
- **const country** cannot be changed once assigned.

Using **const** whenever possible improves code safety and readability.

2. Data Types

Concept

Data types specify the type of value a variable can hold.

JavaScript has two categories:

Primitive Data Types

- String
- Number
- Boolean
- Null
- Undefined
- Symbol
- BigInt

Non-Primitive Data Types

- Object
- Array
- Function

Practical Program

```
let username = "Yash";    // String
let marks = 85;           // Number
let isPassed = true;      // Boolean
let result = null;        // Null
let value;                // Undefined
```

```
console.log(typeof username);
console.log(typeof marks);
console.log(typeof isPassed);
console.log(typeof result);
console.log(typeof value);
```

Explanation

The **typeof operator** helps determine the data type of a variable during runtime. Understanding data types helps prevent logical errors in programs.

3. Loops

Concept

Loops are used to execute a block of code repeatedly until a condition is satisfied.

JavaScript supports several loop types:

- for loop
 - while loop
 - do...while loop
 - for...of loop
 - for...in loop
-

Practical Program

```
// For Loop
for(let i = 1; i <= 5; i++){
  console.log("For Loop:", i);
}

// While Loop
let j = 1;
while(j <= 5){
  console.log("While Loop:", j);
  j++;
}

// Do While Loop
let k = 1;
do{
  console.log("Do While Loop:", k);
  k++;
}while(k <= 5);

// For Of Loop
let fruits = ["Apple", "Banana", "Mango"];
for(let fruit of fruits){
  console.log("Fruit:", fruit);
}

// For In Loop
let student = {name:"Yash", age:21};
for(let key in student){
  console.log(key, student[key]);
}
```

Explanation

Each loop is used for different scenarios:

- **for loop** – when number of iterations is known
 - **while loop** – when condition is evaluated before execution
 - **do...while** – executes at least once
 - **for...of** – iterates through arrays
 - **for...in** – iterates through object properties
-

4. Functions

Concept

Functions are reusable blocks of code that perform specific tasks.

Types of functions:

- Function Declaration
 - Function Expression
 - Arrow Function
 - Callback Function
-

Practical Program

```
// Function Declaration
function greet(name){
    return "Hello " + name;
}

// Function Expression
const add = function(a,b){
    return a + b;
}

// Arrow Function
const multiply = (a,b) => a*b;

// Callback Function
function processUser(name, callback){
    console.log("Processing user:", name);
    callback();
}

function done(){
    console.log("Task Completed");
}

console.log(greet("Yash"));
console.log(add(5,3));
console.log(multiply(4,2));
```

```
processUser("Yash", done);
```

Explanation

Functions allow modular and reusable programming. Callback functions are especially useful in asynchronous operations.

5. Arrays

Concept

Arrays store multiple values in a single variable.

Common Array Methods

- `push()`
 - `pop()`
 - `shift()`
 - `unshift()`
 - `map()`
 - `filter()`
 - `reduce()`
 - `slice()`
 - `splice()`
-

Practical Program

```
let numbers = [1,2,3,4,5];
```

```
numbers.push(6);  
numbers.pop();  
numbers.unshift(0);  
numbers.shift();
```

```
console.log("Array:", numbers);
```

```
// map
let doubled = numbers.map(n => n*2);
console.log("Doubled:", doubled);

// filter
let even = numbers.filter(n => n%2==0);
console.log("Even:", even);

// reduce
let sum = numbers.reduce((acc,val)=>acc+val,0);
console.log("Sum:", sum);
```

Explanation

Array methods allow powerful data manipulation such as transformation, filtering, and aggregation.

6. Objects

Concept

Objects store data in **key-value pairs**.

Objects are useful for representing real-world entities.

Practical Program

```
let student = {
  name: "Yash",
  age: 21,
  course: "Computer Science",

  greet: function(){
    console.log("Hello, my name is " + this.name);
  }
};

console.log(student.name);
console.log(student.age);

student.greet();
```

```
// Object methods
console.log(Object.keys(student));
console.log(Object.values(student));
console.log(Object.entries(student));
```

Explanation

Objects allow structured data storage. Methods like **Object.keys()**, **Object.values()**, and **Object.entries()** help access object information.

7. DOM and Event Handling

Concept

The **DOM (Document Object Model)** allows JavaScript to interact with HTML elements dynamically.

Event handling allows execution of code when users perform actions like clicking buttons.

Practical Program

HTML

```
<button id="btn">Click Me</button>
<p id="message"></p>
```

JavaScript

```
let button = document.getElementById("btn");

button.addEventListener("click", function(){
    document.getElementById("message").innerText = "Button Clicked!";
});
```

Explanation

When the button is clicked, the event listener triggers a function that updates the text in the paragraph element.

8. Promises

Concept

Promises handle asynchronous operations and represent the eventual completion or failure of an operation.

Promise states:

- Pending
 - Resolved
 - Rejected
-

Practical Program

```
let promise = new Promise((resolve, reject)=>{
  let success = true;

  if(success){
    resolve("Operation Successful");
  }else{
    reject("Operation Failed");
  }
});

promise
.then(result => console.log(result))
.catch(error => console.log(error));
```

Explanation

Promises make asynchronous code easier to manage compared to nested callbacks.

9. Async and Await

Concept

Async/Await provides a cleaner way to handle promises.

- **async** makes a function return a promise.
 - **await** pauses execution until promise resolves.
-

Practical Program

```
function fetchData(){
  return new Promise(resolve=>{
    setTimeout(()=>{
      resolve("Data Received");
    },2000);
  });
}
```

```
async function getData(){
  let data = await fetchData();
  console.log(data);
}
```

```
getData();
```

Explanation

Async/Await improves readability and makes asynchronous code look similar to synchronous code.

10. Event Loop

Concept

The **JavaScript Event Loop** is responsible for handling asynchronous operations.

JavaScript runs on a **single-threaded execution model** but can still handle asynchronous tasks using:

- Call Stack
- Web APIs
- Callback Queue
- Event Loop

Working Process

1. Code runs in the **Call Stack**
2. Async tasks move to **Web APIs**
3. When completed, they enter the **Callback Queue**
4. The **Event Loop** pushes them back to the Call Stack

Example

```
console.log("Start");
```

```
setTimeout(()=>{  
  console.log("Async Task");  
},0);
```

```
console.log("End");
```

Output

Start

End

Async Task

Explanation

Even though timeout is **0 milliseconds**, the callback executes later because it is placed in the **event queue**.